DeforestWatch: A Near-Real-Time AI Framework for Automated Deforestation Detection and Alerting Using Multispectral Sentinel-2 Imagery

Aditya Sharma, Lakshay Joshi, Shivam Singh Mahar, Avikshit Trivedi
Department of Computer Science & Engineering, Bhagwan Parshuram Institute of Technology, New Delhi, India

***Abstract*—Acting against deforestation on the ground depends on two things at once: knowing where forest is being lost, and learning about it quickly enough to intervene. Remote-sensing research has tended to treat these as separate problems. The field has produced consistent global forest-change maps, capable segmentation networks, and mature satellite change-detection methods—but these are usually built and tested in isolation from one another. DeforestWatch is a proposed framework that links them into one pipeline. It ingests freely available Sentinel-2 imagery through Google Earth Engine, uses a bi-temporal Normalized Difference Vegetation Index (NDVI) comparison to narrow down where to look, segments the candidate regions with a U-Net, cleans the output with spatial post-processing, and sends automated SMS and email alerts. The model reads four Sentinel-2 bands—Red, Green, Blue, and Near-Infrared—over 256×256 tiles; it is trained with a combined Binary Cross-Entropy and Dice loss under the Adam optimizer at a learning rate of 10-4, and evaluated with Intersection over Union (IoU), F1-score, and pixel accuracy. One point we want to make plainly: the trained model has not yet produced final measurements, and we do not substitute invented numbers for them. What this paper offers instead is the groundwork—a clear statement of the research gap, the system methodology, a comparison framework, and the experimental protocol needed to test whether DeforestWatch is genuinely more useful in operation than the approaches that came before it. The contribution, then, is one of integration: turning satellite observations and a deep-learning segmentation mask into a low-cost, automated, field-oriented alerting workflow.**

***Index Terms*—deforestation detection, remote sensing, Sentinel-2, U-Net, semantic segmentation, NDVI, change detection, Google Earth Engine, automated alerts.**

I. INTRODUCTION

Forests hold up a large share of the terrestrial biosphere. They shelter biodiversity, lock away carbon, regulate water, and support the livelihoods of people who live near them. The Food and Agriculture Organization put the annual rate of deforestation at roughly 10 million hectares over 2015–2020, and noted that the picture varies sharply from region to region, with forest being lost in some places and gained in others [1]. Numbers on that scale are the reason monitoring matters: to be useful for enforcement or conservation, a system has to catch localized changes fast. Satellite remote sensing is what made large-scale forest monitoring feasible in the first place. Hansen et al. built globally consistent maps of forest-cover loss and gain from Earth-observation data at 30 m resolution, and that work still anchors much of the quantitative analysis done today [2]. But a global historical map and an operational alerting system are not the same thing, and they are not trying to do the same job. The former is about consistent, retrospective mapping. The latter has to worry about a different list of problems—how stale the imagery is, whether clouds are in the way, how inference gets automated, where exactly the change happened, how the user gets notified, and whether any of it is affordable to run. Deep learning has become the standard tool for pulling fine spatial detail out of satellite imagery. U-Net, in particular, offers an encoder-decoder design that combines features learned at several spatial scales as it rebuilds its output. That property suits forest-loss mapping well, because cleared areas often have ragged boundaries and may cover only a small slice of a scene. It is no surprise, then, that U-Net and its variants have been studied repeatedly for deforestation mapping with Sentinel-2 and Landsat imagery [3]–[5]. Attention U-Net has posted competitive segmentation scores on Sentinel-2 deforestation data, and other work has lined up several fully convolutional architectures against each other for monitoring the Amazon [4], [5]. Effective as these methods are, accuracy on its own does not describe everything an operational monitor has to get right. A real deployment also has to handle image acquisition, cloud contamination, automated preprocessing, pinning down where a detected change sits, the compute to process it, and getting word out in time. Platforms like Google Earth Engine supply the infrastructure for large-scale geospatial processing [6], and operational services show how remotely sensed data can be turned into something an end user can actually act on [7]. DeforestWatch is built on top of these established pieces—satellite-data processing, deep-learning segmentation, spatial event generation, and automated notification—assembled into one proposed workflow.

A. Research Problem

The question we set out to answer is this: how can freely available multispectral satellite imagery and deep learning be combined into an automated pipeline that detects meaningful forest-cover change, marks the affected areas

down to the pixel, and gets verified events to the people who need them with as little manual work as possible?

B. Research Objectives

Build an automated satellite-data ingestion and preprocessing pipeline on Google Earth Engine. Use bi-temporal NDVI change as a cheap first pass to flag candidate vegetation-loss regions. Apply U-Net semantic segmentation to outline candidate deforestation patches at the pixel level. Turn detected patches into spatial alerts carrying location, area, confidence, and timestamp. Provide a web dashboard together with SMS and email notifications for operational users. Evaluate the approach with IoU, F1-score, precision, recall, pixel accuracy, false-alarm rate, inference time, and alert latency. Compare the system against relevant prior methods under controlled conditions, rather than lining up published numbers that were never meant to be compared directly.

II. RELATED WORK

The work that bears on DeforestWatch falls into five threads that connect to one another: global forest-change mapping, semantic segmentation, multispectral and multi-sensor change detection, scalable geospatial processing, and operational alert delivery. We take each in turn.

A. Global Forest-Change Mapping

Hansen et al. drew on Landsat observations to assemble a globally consistent record of forest-cover loss and gain from 2000 to 2012 at 30 m resolution [2]. It showed what a consistent, satellite-derived forest-change product is worth, and it is still a go-to reference for labeling and validating forest loss. For our purposes, a product like this is valuable as a source of labels and reference data—but it does not, by itself, deliver the end-to-end alerting workflow we are after.

B. U-Net and Pixel-Level Segmentation

Ronneberger et al. introduced U-Net as an encoder-decoder segmentation network that carries information from earlier, higher-resolution stages across to the matching stages of the decoder [3]. That reuse of features lets the network hold onto spatial detail while it rebuilds the segmentation output step by step. Those same qualities make U-Net a sensible baseline for satellite imagery, where getting the boundaries of a forest-loss region right is what counts. We adopt the standard U-Net as a baseline and make no claim to architectural novelty on that front.

C. Deforestation-Specific Deep Learning

Plenty of work has aimed deep learning squarely at deforestation, across different architectures and different parts of the world. Shumilo et al. looked at automated deforestation detection in Ukraine using deep learning and remotely sensed imagery [8]. John and Zhang put Attention U-Net up against several established segmentation models on Sentinel-2 data from the Amazon and the Atlantic Forest, and found Attention U-Net came out ahead in their comparison [4]. Bragagnolo et al. ran a wider comparison of fully convolutional approaches for Amazon deforestation mapping, drawing on both Landsat-8 and Sentinel-2 [5]. Read together, these studies make the point that architecture, sensor, spectral information, and experimental design all leave a mark on how well forest-change detection works. A 2024 systematic review by Md Jelas et al. pulled this literature together, noting the staying power of architectures such as U-Net, DeepLabV3, ResNet, SegNet, and FCN, and picking out multiscale learning, feature fusion, attention, and efficiency as the directions the field keeps returning to [9]. That reinforces our choice to lean on U-Net as a practical baseline, and to leave attention or multi-sensor fusion for later, controlled experiments rather than folding them in untested.

D. Generalization and Change Detection

Pyo et al. studied U-Net-based forest-change segmentation in South Korea and looked hard at how well it generalizes across different forest landscapes and conditions [10]. That question matters to us directly: a model trained in one ecological setting will not necessarily carry over to another. We take the concern seriously in DeforestWatch, and the project documentation proposes transfer learning and region-specific fine-tuning as the fallback when a model trained on, say, Amazon imagery is deployed somewhere else. Within our pipeline, NDVI difference acts as a screening stage that runs before segmentation. The idea is to pick out places where the vegetation signal has dropped sharply, and so keep the neural network from having to chew through areas that are almost certainly fine. It is a cheap thing to compute. What it is not is a fixed ecological constant—the meaning of a given NDVI threshold shifts with season, forest type, atmosphere, and how the image was captured. For that reason we treat the threshold as an experimental parameter to be validated against independently labeled data, not as a universal marker of deforestation.

E. Operational Geospatial Processing and Alerts

Gorelick et al. introduced Google Earth Engine as a cloud environment for planetary-scale geospatial analysis, pairing a large co-located data catalog with the compute to work on it [6]. DeforestWatch uses the GEE Python API to pull and filter imagery on demand, which spares us from hauling large archives down locally. Davies et al. described the FIRMS architecture for pushing satellite-derived fire information out through web services and customized email alerts [7]. FIRMS is about fire, not deforestation—but its operational shape is exactly the pattern we care about: take a satellite observation and put it in front of an end user as something they can use.

III. RESEARCH GAP AND CONTRIBUTION

The literature shows that the individual building blocks of automated forest monitoring have all been worked on,

TABLE I
RESEARCH POSITION OF DEFORESTWATCH

Capability	Prior research focus	DeforestWatch research position
Global change mapping	Large-scale historical forest-loss products	Uses historical products as reference/label sources
Pixel segmentation	U-Net and advanced variants	Uses four-band U-Net as a practical baseline
Change detection	Bi-temporal or multi-temporal remote sensing	Uses NDVI pre-filtering before segmentation
Data processing	Cloud-scale geospatial computation	Uses Google Earth Engine for automated ingestion
Operational delivery	Maps, APIs, or domain-specific alerts	Adds automated SMS/email alerts and dashboard integration
Feedback	Often evaluated as a static model	Includes stakeholder verification and a retraining pathway
Accessibility	May require specialist infrastructure	Targets a low-cost, open-source deployment stack

and worked on thoroughly—but they tend to be evaluated toward different ends and under different experimental setups. Some studies chase segmentation quality; others focus on change detection, large-scale processing, or getting information out to users. What is missing is the seam that joins them. DeforestWatch is proposed as exactly that seam: a system-level combination of these components. So the contribution here is not a new segmentation architecture. It is the workflow: bi-temporal vegetation-change screening, followed by four-band U-Net segmentation, followed by spatial post-processing and automated notification. The experiments we plan are meant to find out whether that particular chain strikes a workable balance among detection quality, compute cost, false alarms, and how long an alert takes to arrive.

IV. PROPOSED DEFORESTWATCH METHODOLOGY

The architecture is organized into five functional layers—data ingestion, preprocessing, AI detection, alert generation, and monitoring/feedback. The design follows the submitted project synopsis and final report.

A. Data Ingestion

Sentinel-2 is our primary source. The mission delivers multispectral imagery that includes 10 m bands for Blue (B2), Green (B3), Red (B4), and Near-Infrared (B8), and its two-satellite constellation revisits a given location roughly every five days when conditions cooperate [11]. DeforestWatch queries Google Earth Engine for imagery over a registered monitoring region and time window. As it stands, the design drops any scene with more than 20% cloud cover before doing anything else with it.

B. Preprocessing and NDVI

Preprocessing handles cloud masking, reflectance normalization, reprojection, and tiling. The U-Net input is a 256×256-pixel tile with four channels: Red, Green, Blue, and Near-Infrared. For the vegetation-change screen, NDVI is computed as

$$\mathrm{NDVI} = \frac{\mathrm{NIR} - \mathrm{Red}}{\mathrm{NIR} + \mathrm{Red}} \tag{1}$$

A baseline image is compared against a more recent one. The current design flags a candidate area when NDVI drops by more than 0.2 between the two, and only those candidates go on to the network. As noted earlier, this 0.2 figure is a hyperparameter to be tuned, not an ecological constant—it should be optimized or checked against a held-out validation set rather than trusted on faith.

C. U-Net Segmentation

Candidate tiles then go to the U-Net. On the way down, the encoder turns the input into progressively higher-level features while shrinking spatial resolution; on the way back up, the decoder rebuilds the prediction at full scale. The information passed across between matching encoder and decoder stages is what preserves the fine spatial detail that boundary-finding depends on [3]. The output is a binary mask that separates pixels judged deforested from pixels judged intact forest. The loss combines Binary Cross-Entropy with Dice loss.

$$L_{\mathrm{total}} = L_{\mathrm{BCE}} + L_{\mathrm{Dice}} \tag{2}$$

Ltotal = LBCE + LDice. The reason is class imbalance—deforested pixels can make up a tiny fraction of a scene next to intact forest, and a plain pixel-wise loss handles that poorly. Training is specified in PyTorch with Adam at a learning rate of 10-4. The project documentation lists targets of IoU > 0.75 and F1 > 0.80. These are goals we are aiming for, not results we have measured.

D. Spatial Post-Processing

The predicted tiles are stitched back into a region-level mask. Connected-component analysis then groups neighboring positive pixels into candidate patches, and the design ignores anything smaller than one hectare. For every patch that clears the bar, the system records its coordinates, estimated area, confidence, and timestamp in a spatial database.

E. Alerting and Dashboard

Once a candidate passes both the area threshold and the model-confidence threshold, the alert engine sends SMS and email notifications through Twilio. The web dashboard is built with React and Leaflet for interactive maps, and a PostgreSQL/PostGIS database keeps the boundaries and the event history. There is a human in the loop as well: stakeholders can mark alerts as true or false positives, and those verified events can feed back into later rounds of retraining.

F. End-to-End Workflow

Register a geographic monitoring region. Retrieve baseline and current Sentinel-2 observations through Google Earth Engine. Apply cloud filtering, cloud masking, normalization, reprojection, and tiling. Compute baseline and current NDVI, and pick out candidate change regions. Run U-Net inference on the candidate tiles. Stitch the tile predictions and run connected-component analysis. Apply the minimum-area and confidence thresholds. Store the event and fire off the SMS/email alerts. Update the interactive dashboard. Collect user verification to feed the next retraining cycle.

V. EXPERIMENTAL DESIGN

Because the trained model has not yet produced a finalized dataset of results, this section lays out how we intend to evaluate the system rather than reporting numbers we cannot yet stand behind. The experiments compare the system's individual components on a shared dataset, a shared preprocessing pipeline, and a shared test protocol. The point of holding everything else constant is to separate whatever improvement comes from NDVI-based candidate selection from whatever comes from the segmentation model itself.

A. Datasets

The plan is to combine the Hansen Global Forest Change dataset with the Kaggle Planet: Understanding the Amazon dataset for training and reference labeling. We intend to split the data at the level of geographic regions—training, validation, and test—so that the same region does not leak across the split. Wherever it is feasible, the test region should sit somewhere geographically distinct from the training regions, so that generalization actually gets tested rather than assumed.

B. Baselines

The most informative controlled comparison needs at least three arms: (1) NDVI thresholding on its own, (2) U-Net with no NDVI pre-filter, and (3) NDVI pre-filtering feeding into U-Net, which is the DeforestWatch configuration. If resources allow, we would also like to reproduce an Attention U-Net baseline, since John and Zhang reported strong results from attention on Sentinel-2 deforestation data [4]. Every baseline gets evaluated on the same test set, under the same preprocessing and the same evaluation rules.

C. Evaluation Metrics

D. Ablation Experiments

An ablation study should show what each major component is actually worth. We plan the following variants: U-Net alone; U-Net + NDVI pre-filter; U-Net + four spectral bands; U-Net + NDVI + four bands; and U-Net + NDVI + four bands + feedback/retraining. Running these keeps us honest—it stops the paper from crediting the whole system for a gain without showing which piece produced it.

TABLE II
Evaluation Metrics

Metric	Purpose	Interpretation
Precision	Correctness of predicted deforestation pixels	Higher is better; matters most for cutting false alarms
Recall	How much real deforestation gets caught	Higher is better; matters most for not missing events
F1-score	Harmonic mean of precision and recall	Balances false positives against false negatives
IoU	Overlap between predicted and reference masks	Higher means better spatial agreement
Pixel accuracy	Fraction of pixels classified correctly	Useful, but can mislead under class imbalance
False-alarm rate	How often alerts are raised in error	Lower is better for operational use
Inference time	Compute cost per tile or region	Lower supports near-real-time operation
Alert latency	Time from usable imagery to notification	Lower improves operational response

TABLE III
Comparison With Existing Approaches

Approach	Primary strength	Key limitation for DeforestWatch's objective
Hansen et al. [2]	Global, consistent forest-change mapping	Historical/global product, not a local automated alert pipeline
U-Net [3]	Strong pixel-level segmentation architecture	Generic architecture; supplies no ingestion or alerting of its own
Attention U-Net [4]	Strong Sentinel-2 deforestation segmentation	Higher complexity; published scores use study-specific datasets
Bragagnolo et al. [5]	Systematic comparison of FCN architectures and sensors	Focuses on segmentation, not end-to-end field alerting
GEE [6]	Scalable planetary geospatial computation	Infrastructure/platform, not a complete deforestation alert model
FIRMS architecture [7]	Demonstrates near-real-time satellite information delivery	Built for active-fire monitoring, not deforestation segmentation
DeforestWatch	Combines ingestion, NDVI screening, U-Net, spatial alerts, notifications	Quantitative performance still pending experimental validation

VI. COMPARISON WITH EXISTING APPROACHES

Numbers reported by different studies are not head-to-head results, and should not be read as such. The underlying experiments differ in imagery, geographic coverage, how the data was annotated, which bands went in, how train and test were split, and which metrics were used. So the published figures below are here for context, not as a claim that DeforestWatch already beats these approaches. A fair quantitative comparison will mean reproducing the chosen baselines on the same test data under the same evaluation procedure.

VII. RESULTS AND DISCUSSION

At this stage of the project, we do not have experimental measurements from the trained DeforestWatch model. This section therefore reports no accuracy, precision, recall, F1-score, IoU, false-alarm rate, or alert latency as achieved results. The IoU and F1 figures in the project documentation are development targets, and reading them as measured performance would be a mistake.

A. Required Results Table

TABLE IV

TABLE IV
Synthetic Demonstration Results - Pair 01

Method	Prec.	Rec.	F1	IoU	Pix. Acc.	Latency
NDVI threshold only	1.00	1.00	1.00	1.00	1.00	116.6 ms
U-Net only	N/A	N/A	N/A	N/A	N/A	Not run
NDVI + U-Net	N/A	N/A	N/A	N/A	N/A	Not run
Attention U-Net	N/A	N/A	N/A	N/A	N/A	Not run

Pair 01 uses synthetic ground truth; neural-network rows were not run and must not be read as measured accuracy.

Once the experiments are done, the first hypothesis to test is that NDVI pre-filtering can cut down on wasted U-Net inference while still holding onto high recall for genuine forest loss. The second is that four-band multispectral input sharpens boundary discrimination compared with RGB alone. The third is operational: that spatial post-processing plus automated notification shortens alert latency against any workflow that still needs a human to review each acquisition by hand. The discussion that matters most will have to weigh statistical performance against operational performance together. A model with a slightly better F1 but a much heavier compute bill may be the wrong choice for a low-resource deployment. And the reverse holds too—a small drop in segmentation score can be a fair trade if the system raises fewer false alarms, runs faster, and reliably gets an actionable alert out the door.

VIII. LIMITATIONS

This work has real limits, and it is worth naming them. First, without final measurements we cannot make quantitative claims about being better than anything. Second, Sentinel-2 is an optical sensor, so it stays vulnerable to cloud and atmospheric contamination; the documentation proposes cloud masking and, down the line, a SAR fallback, but the SAR side has not been validated experimentally. Third, generalization is a genuine worry. A model trained mostly on Amazon imagery may not transfer cleanly to forests with different species mixes, seasonality, topography, farming patterns, or atmospheric conditions. Fourth, a 10 m pixel can miss very small clearings, and the one-hectare alert threshold deliberately suppresses smaller patches. Finally, NDVI thresholds can throw false positives when vegetation shifts with the season or the environment, which is exactly why the fixed 0.2 value needs validating rather than being treated as optimal everywhere.

IX. FUTURE WORK

Finish the controlled experiments and report confidence intervals or repeated-run statistics. Test NDVI pre-filtering against direct segmentation on identical test data. Reproduce Attention U-Net and other lightweight architectures as controlled baselines. Add Sentinel-1 SAR fusion for cloud-prone regions. Evaluate transfer learning and region-specific fine-tuning across several forest biomes. Swap the single NDVI threshold for a multi-temporal change model that accounts for seasonal vegetation cycles. Measure end-to-end alert latency from image availability to notification delivery. Look at false-alarm burden through the eyes of field users, not just pixel-level metrics. Investigate model compression and quantization for low-cost cloud or edge inference. Build an auditable feedback loop in which verified alerts are versioned and used for periodic retraining.

X. CONCLUSION

We have proposed DeforestWatch as an integrated framework for automated forest-loss monitoring built on freely available satellite imagery and deep-learning segmentation. The workflow ties together Sentinel-2 acquisition, vegetation-change screening, U-Net pixel classification, spatial event processing, and automated communication of what it finds. Rather than proposing yet another segmentation architecture, we ask a different question: whether these established techniques, arranged in the right order, add up to a monitoring pipeline that works in practice. Whether the system actually delivers is still an open question, and an experimental one. The planned evaluation is designed to find out whether NDVI-based candidate selection can trim unnecessary inference without letting genuine forest loss slip through, and to measure segmentation quality, false alarms, compute cost, and end-to-end latency alongside it. Until those experiments are run, DeforestWatch is best understood as a proposed methodology—not a validated claim to be better.

References

[1] Food and Agriculture Organization of the United Nations, "Global Forest Resources Assessment 2020," FAO, Rome, Italy, 2020.

[2] M. C. Hansen, P. V. Potapov, R. Moore, M. Hancher, S. A. Turubanova, A. Tyukavina, D. Thau, S. V. Stehman, S. J. Goetz, T. R. Loveland, A. Kommareddy, A. Egorov, L. Chini, C. O. Justice, and J. R. G. Townshend, "High-resolution global maps of 21st-century forest cover change," Science, vol. 342, no. 6160, pp. 850–853, 2013, doi: 10.1126/science.1244693.

[3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in Proc. MICCAI, 2015, pp. 234–241, doi: 10.1007/978-3-319-24574-4_28.

[4] D. John and C. Zhang, "An attention-based U-Net for detecting deforestation within satellite sensor imagery," International Journal of Applied Earth Observation and Geoinformation, vol. 107, Art. no. 102685, 2022, doi: 10.1016/j.jag.2022.102685.

[5] L. Bragagnolo, A. da Silva, and others, "Deforestation detection with fully convolutional networks in the Amazon Forest from Landsat-8 and Sentinel-2 images," Remote Sensing, vol. 13, no. 24, Art. no. 5084, 2021, doi: 10.3390/rs13245084.

[6] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore, "Google Earth Engine: Planetary-scale geospatial analysis for everyone," Remote Sensing of Environment, vol. 202, pp. 18–27, 2017, doi: 10.1016/j.rse.2017.06.031.

[7] D. K. Davies, S. Ilavajhala, M. M. Wong, and C. O. Justice, "Fire Information for Resource Management System: Archiving and distributing MODIS active fire data," IEEE Trans. Geosci. Remote Sens., vol. 47, no. 1, pp. 72–79, 2009, doi: 10.1109/TGRS.2008.2002076.

[8] L. Shumilo, M. Lavreniuk, N. Kussul, and B. Shevchuk, "Automatic deforestation detection based on the deep learning in Ukraine," in Proc. 11th IEEE Int. Conf. Intelligent Data Acquisition and Advanced Computing Systems (IDAACS), 2021, vol. 1, pp. 337–342, doi: 10.1109/IDAACS53288.2021.9661008.

[9] I. Md Jelas, M. A. Zulkifley, M. Abdullah, and M. Spraggon, "Deforestation detection using deep learning-based semantic segmentation techniques: A systematic review," Frontiers in Forests and Global Change, vol. 7, Art. no. 1300060, 2024, doi: 10.3389/ffgc.2024.1300060.

[10] J. Pyo, K.-j. Han, Y. Cho, D. Kim, and D. Jin, "Generalization of U-Net semantic segmentation for forest change detection in South Korea using airborne imagery," Forests, vol. 13, no. 12, Art. no. 2170, 2022, doi: 10.3390/f13122170.

[11] European Space Agency, "Sentinel-2 User Handbook," ESA Standard Document, Issue 1, Revision 2, 2015.

[12] United Nations, "Transforming our world: The 2030 Agenda for Sustainable Development," UN General Assembly Resolution A/RES/70/1, 2015.